

ECEN 662: Estimation & Detection Theory

Lecture 18: Monte Carlo Integration and Importance Sampling

Tie Liu

Texas A&M University

Classical Monte Carlo integration

- In previous lecture, we discussed the problem of *sampling* from a known distribution

Classical Monte Carlo integration

- In previous lecture, we discussed the problem of *sampling* from a known distribution
- An important application of sampling is to evaluate *hard-to-evaluate* expectations/integrals

Classical Monte Carlo integration

- In previous lecture, we discussed the problem of *sampling* from a known distribution
- An important application of sampling is to evaluate *hard-to-evaluate* expectations/integrals
- Consider the generic problem of evaluating the expectation/integral

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

Classical Monte Carlo integration

- In previous lecture, we discussed the problem of *sampling* from a known distribution
- An important application of sampling is to evaluate *hard-to-evaluate* expectations/integrals
- Consider the generic problem of evaluating the expectation/integral

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

- For cases where the above expectation/integral *cannot* be evaluated analytically, it is natural to propose using multiple *independent* samples $(X_1, \dots, X_m)$ drawn from the density f to approximate it:

$$\hat{h}_m = \frac{1}{m} \sum_{i=1}^m h(X_i)$$

- This approach is often referred to as the *Monte Carlo* method

Challenges

- There are two main challenges for applying the aforementioned Monte Carlo integration

Challenges

- There are two main challenges for applying the aforementioned Monte Carlo integration
- First, it may not be easy to sample from the density f , especially it is *high-dimensional*

Challenges

- There are two main challenges for applying the aforementioned Monte Carlo integration
- First, it may not be easy to sample from the density f , especially it is *high-dimensional*
- Second, by the *weak law of large numbers*,

$$\hat{h}_m \xrightarrow{i.p.} \mathbb{E}_f[h(X)]$$

in the limit as $m \rightarrow \infty$, which is the main justification of the Monte Carlo method

Challenges

- There are two main challenges for applying the aforementioned Monte Carlo integration
- First, it may not be easy to sample from the density f , especially it is *high-dimensional*
- Second, by the *weak law of large numbers*,

$$\hat{h}_m \xrightarrow{i.p.} \mathbb{E}_f[h(X)]$$

in the limit as $m \rightarrow \infty$, which is the main justification of the Monte Carlo method

- In practice, however, the sample size m is always *finite*. In order for the method to work, the *variance* of the estimate

$$\text{Var}_f[\hat{h}_m] = \frac{\text{Var}_f[h(X)]}{m}$$

has to be *sufficiently small*

Rare event estimation

- The issue of variance is particularly problematic when there is a significant “*mismatch*” between h and f

Rare event estimation

- The issue of variance is particularly problematic when there is a significant “*mismatch*” between h and f
- One such example is

$$h(x) = 1_{\{x \in B\}}$$

where B is a *rare* event under f

Rare event estimation

- The issue of variance is particularly problematic when there is a significant “*mismatch*” between h and f

- One such example is

$$h(x) = 1_{\{x \in B\}}$$

where B is a *rare* event under f

- In this case, $h(X) \sim \mathcal{B}(p_B)$, where

$$p_B = \mathbb{P}_f(B)$$

so

$$\mathbb{E}_f[h(X)] = p_B \quad \text{and} \quad \text{Var}_f[\hat{h}_m] = \frac{p_B - p_B^2}{m}$$

Rare event estimation

- The issue of variance is particularly problematic when there is a significant “*mismatch*” between h and f
- One such example is

$$h(x) = 1_{\{x \in B\}}$$

where B is a *rare* event under f

- In this case, $h(X) \sim \mathcal{B}(p_B)$, where

$$p_B = \mathbb{P}_f(B)$$

so

$$\mathbb{E}_f[h(X)] = p_B \quad \text{and} \quad \text{Var}_f[\hat{h}_m] = \frac{p_B - p_B^2}{m}$$

- If we require the standard deviation of $\hat{h}_m$ to be in the *same order* as p_B , we need

$$\sqrt{\frac{p_B - p_B^2}{m}} \approx p_B \quad \implies \quad m \approx \frac{1}{p_B} - 1$$

which can be exceedingly *large* for a very *rare* event B

Importance sampling

- The method of *importance sampling* is an evaluation of the expectation/integral

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

based on samples from a *proposal* distribution g (instead of directly sampling from f)

Importance sampling

- The method of *importance sampling* is an evaluation of the expectation/integral

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

based on samples from a *proposal* distribution g (instead of directly sampling from f)

- The method is based on the following alternative representation of the expectation/integral:

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)\omega(x)g(x)dx = \mathbb{E}_g[\omega(X)h(X)]$$

where $\omega(x) = f(x)/g(x)$ is the density ratio

Importance sampling

- The method of *importance sampling* is an evaluation of the expectation/integral

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

based on samples from a *proposal* distribution g (instead of directly sampling from f)

- The method is based on the following alternative representation of the expectation/integral:

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)\omega(x)g(x)dx = \mathbb{E}_g[\omega(X)h(X)]$$

where $\omega(x) = f(x)/g(x)$ is the density ratio

- The above representation immediately leads to the following approximation based on samples $(X_1, \dots, X_m)$ drawn from g :

$$\hat{h}_m = \frac{1}{m} \sum_{i=1}^m \omega(X_i)h(X_i)$$

- The aforementioned importance sampling method is a manifest to the fact that a given integral is *not* intrinsically associated with a given distribution

- The aforementioned importance sampling method is a manifest to the fact that a given integral is *not* intrinsically associated with a given distribution
- Importance sampling is thus of considerable interest because it puts very *little* restriction on the choice of the proposal distribution, which can be chosen from distributions that are easy to sample from

- The aforementioned importance sampling method is a manifest to the fact that a given integral is *not* intrinsically associated with a given distribution
- Importance sampling is thus of considerable interest because it puts very *little* restriction on the choice of the proposal distribution, which can be chosen from distributions that are easy to sample from
- Moreover, the same samples from g can be used *repeatedly* for different functions h but also for different densities f

Variance control

- First note that the variance of the importance sampling estimator is *finite* only when the expectation

$$\mathbb{E}_g[(\omega(X)h(X))^2] < \infty$$

Variance control

- First note that the variance of the importance sampling estimator is *finite* only when the expectation

$$\mathbb{E}_g[(\omega(X)h(X))^2] < \infty$$

- Thus, proposal distributions with tails *lighter* than those of f (that is, those with *unbounded* density ratio $\omega(x)$) are *not* appropriate for importance sampling

Variance control

- First note that the variance of the importance sampling estimator is *finite* only when the expectation

$$\mathbb{E}_g[(\omega(X)h(X))^2] < \infty$$

- Thus, proposal distributions with tails *lighter* than those of f (that is, those with *unbounded* density ratio $\omega(x)$) are *not* appropriate for importance sampling
- In particular, if the density ratio $\omega(x)$ is *bounded*, the *acceptance-rejection* algorithm also applies. A more detailed comparison between importance sampling and rejection sampling will be discussed later in this lecture

Variance control

- First note that the variance of the importance sampling estimator is *finite* only when the expectation

$$\mathbb{E}_g[(\omega(X)h(X))^2] < \infty$$

- Thus, proposal distributions with tails *lighter* than those of f (that is, those with *unbounded* density ratio $\omega(x)$) are *not* appropriate for importance sampling
- In particular, if the density ratio $\omega(x)$ is *bounded*, the *acceptance-rejection* algorithm also applies. A more detailed comparison between importance sampling and rejection sampling will be discussed later in this lecture
- Among proposal distributions leading to finite variances for the importance sampling estimator, it is possible to identify *optimal* proposal distribution corresponding to a given function h and a fixed distribution f

- More specifically, the choice of g that minimizes the *variance* of the importance sampling estimator is

$$g^*(x) \propto_x |h(x)|f(x)$$

- More specifically, the choice of g that minimizes the *variance* of the importance sampling estimator is

$$g^*(x) \propto_x |h(x)|f(x)$$

- From a practical point of view, the above result suggests looking for proposal distributions g for which $\omega(x)|h(x)|$ is almost *constant*

Self-normalizing importance sampling

- An alternative to the vanilla importance sampling estimator

$$\hat{h}_m = \frac{1}{m} \sum_{i=1}^m \omega(X_i) h(X_i)$$

is the following *self-normalizing importance sampling* estimator

$$\hat{h}'_m = \frac{\sum_{i=1}^m \omega(X_i) h(X_i)}{\sum_{i=1}^m \omega(X_i)}$$

Self-normalizing importance sampling

- An alternative to the vanilla importance sampling estimator

$$\hat{h}_m = \frac{1}{m} \sum_{i=1}^m \omega(X_i) h(X_i)$$

is the following *self-normalizing importance sampling* estimator

$$\hat{h}'_m = \frac{\sum_{i=1}^m \omega(X_i) h(X_i)}{\sum_{i=1}^m \omega(X_i)}$$

- Note that as $m \rightarrow \infty$,

$$\frac{1}{m} \sum_{i=1}^m \omega(X_i) \stackrel{i.p.}{\approx} \mathbb{E}_g[\omega(X)] = \int_{\mathcal{X}} \frac{f(x)}{g(x)} g(x) dx = \int_{\mathcal{X}} f(x) dx = 1$$

Self-normalizing importance sampling

- An alternative to the vanilla importance sampling estimator

$$\hat{h}_m = \frac{1}{m} \sum_{i=1}^m \omega(X_i) h(X_i)$$

is the following *self-normalizing importance sampling* estimator

$$\hat{h}'_m = \frac{\sum_{i=1}^m \omega(X_i) h(X_i)}{\sum_{i=1}^m \omega(X_i)}$$

- Note that as $m \rightarrow \infty$,

$$\frac{1}{m} \sum_{i=1}^m \omega(X_i) \stackrel{i.p.}{\sim} \mathbb{E}_g[\omega(X)] = \int_{\mathcal{X}} \frac{f(x)}{g(x)} g(x) dx = \int_{\mathcal{X}} f(x) dx = 1$$

- Therefore, just like the vanilla importance sampling estimator, the *self-normalizing* importance sampling estimator is also a *consistent* estimator of the expectation $\mathbb{E}_f[h(X)]$

- In addition, the *self-normalizing* importance sampling estimator also enjoys a couple of important additional benefits

- In addition, the *self-normalizing* importance sampling estimator also enjoys a couple of important additional benefits
- First, unlike the vanilla importance sampling estimator, the *self-normalizing* importance sampling estimator is slightly *biased* and thus may enjoy the benefit of having a *smaller* variance

- In addition, the *self-normalizing* importance sampling estimator also enjoys a couple of important additional benefits
- First, unlike the vanilla importance sampling estimator, the *self-normalizing* importance sampling estimator is slightly *biased* and thus may enjoy the benefit of having a *smaller* variance
- Second, unlike the vanilla importance sampling estimator, which requires precise knowledge of the density f , the *self-normalizing* importance sampling estimator can be implemented based on an *un-normalized* density $\tilde{f}$ with an *unknown* scaling factor, which makes it particularly appealing when f is a *posterior* distribution in Bayesian statistics

Monte Carlo marginalization

- Consider the problem of computing the marginal density $f(x)$ from the joint density $f(x, z)$ via *importance sampling*:

$$f(x) = \int_{\mathcal{Z}} f(x, z) dz = \int_{\mathcal{Z}} \frac{f(x, z)}{g(z)} g(z) dz$$

where g is the proposal distribution

Monte Carlo marginalization

- Consider the problem of computing the marginal density $f(x)$ from the joint density $f(x, z)$ via *importance sampling*:

$$f(x) = \int_{\mathcal{Z}} f(x, z) dz = \int_{\mathcal{Z}} \frac{f(x, z)}{g(z)} g(z) dz$$

where g is the proposal distribution

- Based on the previous analysis, the *optimal* proposal distribution that minimizes the variance must ensure that the ratio $f(x, z)/g(z)$ is *constant* with respect to z :

$$g(z) \propto_z f(x, z)$$

and hence

$$g(z) = f(z|x)$$

Monte Carlo marginalization

- Consider the problem of computing the marginal density $f(x)$ from the joint density $f(x, z)$ via *importance sampling*:

$$f(x) = \int_{\mathcal{Z}} f(x, z) dz = \int_{\mathcal{Z}} \frac{f(x, z)}{g(z)} g(z) dz$$

where g is the proposal distribution

- Based on the previous analysis, the *optimal* proposal distribution that minimizes the variance must ensure that the ratio $f(x, z)/g(z)$ is *constant* with respect to z :

$$g(z) \propto_z f(x, z)$$

and hence

$$g(z) = f(z|x)$$

- However, knowing the joint density $f(x, z)$, computing the posterior distribution $f(z|x)$ is the *same* as computing the marginal distribution $f(x)$

- Therefore, to perform *marginalization* using importance sampling, one may wish to find an *easy-to-sample-from* and *easy-to-evaluate* proposal distribution that also *approximates* the posterior distribution

- Therefore, to perform *marginalization* using importance sampling, one may wish to find an *easy-to-sample-from* and *easy-to-evaluate* proposal distribution that also *approximates* the posterior distribution
- This is the essential idea of *variational auto-encoding (VAE)*

Variational auto-encoder (VAE)

- Consider a *generative process* that transforms a random source $\mathbf{z}$ into a data sample $\mathbf{x}$ through a one-step *generator*

Variational auto-encoder (VAE)

- Consider a *generative process* that transforms a random source $\mathbf{z}$ into a data sample $\mathbf{x}$ through a one-step *generator*
- The random source $\mathbf{z}$ usually has a *fixed* distribution $p(\mathbf{z})$, which must be easy to draw from

Variational auto-encoder (VAE)

- Consider a *generative process* that transforms a random source $\mathbf{z}$ into a data sample $\mathbf{x}$ through a one-step *generator*
- The random source $\mathbf{z}$ usually has a *fixed* distribution $p(\mathbf{z})$, which must be easy to draw from
- The usual choice for the random source $\mathbf{z}$ is a *standard Gaussian* vector

Variational auto-encoder (VAE)

- Consider a *generative process* that transforms a random source $\mathbf{z}$ into a data sample $\mathbf{x}$ through a one-step *generator*
- The random source $\mathbf{z}$ usually has a *fixed* distribution $p(\mathbf{z})$, which must be easy to draw from
- The usual choice for the random source $\mathbf{z}$ is a *standard Gaussian* vector
- The generator can be a *deterministic* function $\mathbf{x} = f_{\theta}(\mathbf{z})$ but more generally a *conditional distribution* $p_{\theta}(\mathbf{x}|\mathbf{z})$

Variational auto-encoder (VAE)

- Consider a *generative process* that transforms a random source $\mathbf{z}$ into a data sample $\mathbf{x}$ through a one-step *generator*
- The random source $\mathbf{z}$ usually has a *fixed* distribution $p(\mathbf{z})$, which must be easy to draw from
- The usual choice for the random source $\mathbf{z}$ is a *standard Gaussian* vector
- The generator can be a *deterministic* function $\mathbf{x} = f_{\theta}(\mathbf{z})$ but more generally a *conditional distribution* $p_{\theta}(\mathbf{x}|\mathbf{z})$
- The conditional distribution $p_{\theta}(\mathbf{x}|\mathbf{z})$ must also be *easy* to draw from

Variational auto-encoder (VAE)

- Consider a *generative process* that transforms a random source $\mathbf{z}$ into a data sample $\mathbf{x}$ through a one-step *generator*
- The random source $\mathbf{z}$ usually has a *fixed* distribution $p(\mathbf{z})$, which must be easy to draw from
- The usual choice for the random source $\mathbf{z}$ is a *standard Gaussian* vector
- The generator can be a *deterministic* function $\mathbf{x} = f_{\theta}(\mathbf{z})$ but more generally a *conditional distribution* $p_{\theta}(\mathbf{x}|\mathbf{z})$
- The conditional distribution $p_{\theta}(\mathbf{x}|\mathbf{z})$ must also be *easy* to draw from
- The generative model $p_{\theta}(\mathbf{x})$ is *implicitly* defined as the *marginal* distribution of $\mathbf{x}$:

$$p_{\theta}(\mathbf{x}) = \int p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})d\mathbf{z} = \mathbb{E}_{\mathbf{Z} \sim p(\mathbf{z})}[p_{\theta}(\mathbf{x}|\mathbf{Z})]$$

Model training

- The goal of learning is thus to identify a model parameter θ such that the generative model

$$p_{\theta} \approx p$$

Model training

- The goal of learning is thus to identify a model parameter θ such that the generative model

$$p_{\theta} \approx p$$

- In statistics, one often use the so-called *K-L divergence* to measure the “distance” between two probability distributions p and q :

$$D_{KL}(p\|q) := \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{X \sim p(x)} \left[\log \frac{p(X)}{q(X)} \right]$$

Model training

- The goal of learning is thus to identify a model parameter θ such that the generative model

$$p_{\theta} \approx p$$

- In statistics, one often use the so-called *K-L divergence* to measure the “distance” between two probability distributions p and q :

$$D_{KL}(p\|q) := \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{X \sim p(X)} \left[\log \frac{p(X)}{q(X)} \right]$$

- One possibility is to learn a parameter θ by minimizing the K-L divergence

$$\begin{aligned} D_{KL}(p\|p_{\theta}) &= \mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} \left[\log \frac{p(\mathbf{X})}{p_{\theta}(\mathbf{X})} \right] \\ &= \mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} [\log p(\mathbf{X})] - \mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} [\log p_{\theta}(\mathbf{X})] \end{aligned}$$

Maximum-likelihood (ML) estimate

- Note that minimizing the K-L divergence $D_{KL}(p\|p_\theta)$ with respect to θ is equivalent to maximizing

$$\mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} [\log p_\theta(\mathbf{X})] \approx \frac{1}{m} \sum_{i=1}^m \log p_\theta(\mathbf{x}_i)$$

with respect to θ

Maximum-likelihood (ML) estimate

- Note that minimizing the K-L divergence $D_{KL}(p||p_\theta)$ with respect to θ is equivalent to maximizing

$$\mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} [\log p_\theta(\mathbf{X})] \approx \frac{1}{m} \sum_{i=1}^m \log p_\theta(\mathbf{x}_i)$$

with respect to θ

- The above approach for learning the model parameter θ is known as the *maximum-likelihood* estimate

Maximum-likelihood (ML) estimate

- Note that minimizing the K-L divergence $D_{KL}(p||p_\theta)$ with respect to θ is equivalent to maximizing

$$\mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} [\log p_\theta(\mathbf{X})] \approx \frac{1}{m} \sum_{i=1}^m \log p_\theta(\mathbf{x}_i)$$

with respect to θ

- The above approach for learning the model parameter θ is known as the *maximum-likelihood* estimate
- To compute the maximum-likelihood estimate, we need to evaluate the marginal distribution

$$p_\theta(\mathbf{x}) = \int p(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z}) d\mathbf{z} = \mathbb{E}_{\mathbf{Z} \sim p(\mathbf{z})} [p_\theta(\mathbf{x}|\mathbf{Z})]$$

for any given $\mathbf{x}$

Maximum-likelihood (ML) estimate

- Note that minimizing the K-L divergence $D_{KL}(p\|p_\theta)$ with respect to θ is equivalent to maximizing

$$\mathbb{E}_{\mathbf{X} \sim p(\mathbf{x})} [\log p_\theta(\mathbf{X})] \approx \frac{1}{m} \sum_{i=1}^m \log p_\theta(\mathbf{x}_i)$$

with respect to θ

- The above approach for learning the model parameter θ is known as the *maximum-likelihood* estimate
- To compute the maximum-likelihood estimate, we need to evaluate the marginal distribution

$$p_\theta(\mathbf{x}) = \int p(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z}) d\mathbf{z} = \mathbb{E}_{\mathbf{Z} \sim p(\mathbf{z})} [p_\theta(\mathbf{x}|\mathbf{Z})]$$

for any given $\mathbf{x}$

- Note, however, that the integral for evaluating $p_\theta(\mathbf{x})$ *cannot* be evaluated analytically

Monte-Carlo (MC) integration

- In practice, one may *estimate* $p_\theta(\mathbf{x})$ through *Monte Carlo integration*:

$$p_\theta(\mathbf{x}) \approx \frac{1}{n} \sum_{j=1}^n p_\theta(\mathbf{x} | \mathbf{z}_j)$$

where $(\mathbf{z}_1, \dots, \mathbf{z}_n)$ are independent samples of the random source $\mathbf{z}$

Monte-Carlo (MC) integration

- In practice, one may *estimate* $p_\theta(\mathbf{x})$ through *Monte Carlo integration*:

$$p_\theta(\mathbf{x}) \approx \frac{1}{n} \sum_{j=1}^n p_\theta(\mathbf{x}|\mathbf{z}_j)$$

where $(\mathbf{z}_1, \dots, \mathbf{z}_n)$ are independent samples of the random source $\mathbf{z}$

- However, the conditional probability $p_\theta(\mathbf{x}|\mathbf{z})$ usually *fluctuates* significantly for different values of $\mathbf{z}$. As a result, the above estimate only becomes reliable when the number of samples drawn from the random source $\mathbf{z}$ becomes very large

Importance sampling to the rescue

- In statistics, there is a clever approach for solving this problem known as *importance sampling*

Importance sampling to the rescue

- In statistics, there is a clever approach for solving this problem known as *importance sampling*
- The main idea is to introduce a new *sampling distribution* $q(\mathbf{z})$ to rewrite the expression for $p_\theta(\mathbf{x})$:

$$\begin{aligned} p_\theta(\mathbf{x}) &= \int p(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z}) d\mathbf{z} \\ &= \int \frac{p(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z})}{q(\mathbf{z})} q(\mathbf{z}) d\mathbf{z} \\ &= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z})} \left[\frac{p(\mathbf{Z}) p_\theta(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z})} \right] \\ &\approx \frac{1}{n} \sum_{j=1}^n \frac{p(\mathbf{z}_j) p_\theta(\mathbf{x}|\mathbf{z}_j)}{q(\mathbf{z}_j)} \end{aligned}$$

where $(\mathbf{z}_1, \dots, \mathbf{z}_n)$ are independent samples drawn according to $q(\mathbf{z})$

Ideal sampling distribution

- Note that the sampling distribution $q(\mathbf{z})$ needs to be both easy to draw samples from and easy to evaluate

Ideal sampling distribution

- Note that the sampling distribution $q(\mathbf{z})$ needs to be both easy to draw samples from and easy to evaluate
- As discussed previously, the *ideal* sampling distribution is one that makes the expectant

$$\frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{q(\mathbf{z})}$$

a *constant* function of $\mathbf{z}$, i.e., *no* fluctuations with respect to $\mathbf{z}$

Ideal sampling distribution

- Note that the sampling distribution $q(\mathbf{z})$ needs to be both easy to draw samples from and easy to evaluate
- As discussed previously, the *ideal* sampling distribution is one that makes the expectant

$$\frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{q(\mathbf{z})}$$

a *constant* function of $\mathbf{z}$, i.e., *no* fluctuations with respect to $\mathbf{z}$

- Letting

$$\frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{q(\mathbf{z})} = c$$

gives

$$q(\mathbf{z}) = \frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{c}$$

where the constant c needs to be chosen such that

$$\int q(\mathbf{z})d\mathbf{z} = 1$$

- This gives

$$c = \int p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})d\mathbf{z} = p_{\theta}(\mathbf{x})$$

and hence

$$q(\mathbf{z}) = \frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{p_{\theta}(\mathbf{x})} = p_{\theta}(\mathbf{z}|\mathbf{x})$$

- This gives

$$c = \int p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})d\mathbf{z} = p_{\theta}(\mathbf{x})$$

and hence

$$q(\mathbf{z}) = \frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{p_{\theta}(\mathbf{x})} = p_{\theta}(\mathbf{z}|\mathbf{x})$$

- That is, the *ideal* sampling distribution is, in fact, given by the *posterior* distribution of $\mathbf{z}$ given $\mathbf{x}$

- This gives

$$c = \int p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})d\mathbf{z} = p_{\theta}(\mathbf{x})$$

and hence

$$q(\mathbf{z}) = \frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{p_{\theta}(\mathbf{x})} = p_{\theta}(\mathbf{z}|\mathbf{x})$$

- That is, the *ideal* sampling distribution is, in fact, given by the *posterior* distribution of $\mathbf{z}$ given $\mathbf{x}$
- Unfortunately, the posterior distribution $p_{\theta}(\mathbf{z}|\mathbf{x})$ is *untenable* because evaluating it requires the knowledge of the marginal distribution $p_{\theta}(\mathbf{x})$

- This gives

$$c = \int p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})d\mathbf{z} = p_{\theta}(\mathbf{x})$$

and hence

$$q(\mathbf{z}) = \frac{p(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})}{p_{\theta}(\mathbf{x})} = p_{\theta}(\mathbf{z}|\mathbf{x})$$

- That is, the *ideal* sampling distribution is, in fact, given by the *posterior* distribution of $\mathbf{z}$ given $\mathbf{x}$
- Unfortunately, the posterior distribution $p_{\theta}(\mathbf{z}|\mathbf{x})$ is *untenable* because evaluating it requires the knowledge of the marginal distribution $p_{\theta}(\mathbf{x})$
- In practice, one may introduce a second parameterized model $q_{\phi}(\mathbf{z}|\mathbf{x})$ to *approximate* the posterior distribution $p_{\theta}(\mathbf{z}|\mathbf{x})$

The encoder and decoder

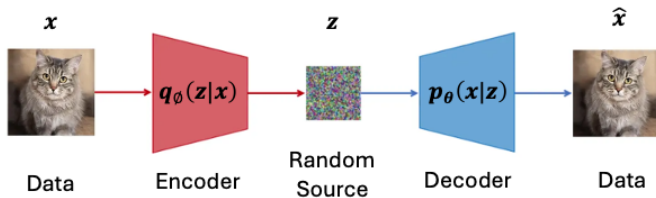
- To summarize, even though our original goal was simply to train a generator $p_{\theta}(\mathbf{x}|\mathbf{z})$ via maximum-likelihood estimate, estimating the marginal likelihood $p_{\theta}(\mathbf{x})$ requires training an additional sampling distribution $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x})$

The encoder and decoder

- To summarize, even though our original goal was simply to train a generator $p_{\theta}(\mathbf{x}|\mathbf{z})$ via maximum-likelihood estimate, estimating the marginal likelihood $p_{\theta}(\mathbf{x})$ requires training an additional sampling distribution $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x})$
- Further note that while the generator $p_{\theta}(\mathbf{x}|\mathbf{z})$ *generates* data from random source, the goal of the sampling distribution $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x})$ is to *recover* the random source from the data

The encoder and decoder

- To summarize, even though our original goal was simply to train a generator $p_{\theta}(\mathbf{x}|\mathbf{z})$ via maximum-likelihood estimate, estimating the marginal likelihood $p_{\theta}(\mathbf{x})$ requires training an additional sampling distribution $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x})$
- Further note that while the generator $p_{\theta}(\mathbf{x}|\mathbf{z})$ *generates* data from random source, the goal of the sampling distribution $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x})$ is to *recover* the random source from the data
- We can think of the role of generator and sampling distribution as a process of first transforming data into a random source using $q_{\phi}(\mathbf{z}|\mathbf{x})$ and then recovering the data using the generator $p_{\theta}(\mathbf{x}|\mathbf{z})$



- This process of *data self-recovery* is usually known as *auto-encoding*. Under the context of *auto-encoding*, we call the sampling distribution $q_\phi(\mathbf{z}|\mathbf{x})$ the *encoder* and the generator $p_\theta(\mathbf{x}|\mathbf{z})$ the *decoder*

- This process of *data self-recovery* is usually known as *auto-encoding*. Under the context of *auto-encoding*, we call the sampling distribution $q_\phi(\mathbf{z}|\mathbf{x})$ the *encoder* and the generator $p_\theta(\mathbf{x}|\mathbf{z})$ the *decoder*
- It is important to note that the aforementioned auto-encoding is *stochastic* in nature in that the recovered data is only equivalent to the original data *statistically* and in general is *not* the same as the original data

Joint training of encoder and decoder

- To jointly learn the encoder and decoder, let us consider the following *variational* representation of $\log p_{\theta}(\mathbf{x})$:

$$\log p_{\theta}(\mathbf{x}) = \max_{q(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}) - D_{KL}(q(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z}|\mathbf{x}))]$$

Joint training of encoder and decoder

- To jointly learn the encoder and decoder, let us consider the following *variational* representation of $\log p_\theta(\mathbf{x})$:

$$\log p_\theta(\mathbf{x}) = \max_{q(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}) - D_{KL}(q(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x}))]$$

- The above representation holds due to the fact that the K-L divergence

$$D_{KL}(q(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x})) \geq 0$$

with the equality holds if and only if

$$q(\cdot|\mathbf{x}) = p_\theta(\cdot|\mathbf{x})$$

Joint training of encoder and decoder

- To jointly learn the encoder and decoder, let us consider the following *variational* representation of $\log p_\theta(\mathbf{x})$:

$$\log p_\theta(\mathbf{x}) = \max_{q(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}) - D_{KL}(q(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x}))]$$

- The above representation holds due to the fact that the K-L divergence

$$D_{KL}(q(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x})) \geq 0$$

with the equality holds if and only if

$$q(\cdot|\mathbf{x}) = p_\theta(\cdot|\mathbf{x})$$

- Note that for any given $\mathbf{x}$,

A variational characterization of the log likelihood

$$\begin{aligned}\log p_\theta(\mathbf{x}) - D_{KL}(q(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x})) \\&= \log p_\theta(\mathbf{x}) - \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{q(\mathbf{Z}|\mathbf{x})}{p_\theta(\mathbf{Z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x})] - \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{q(\mathbf{Z}|\mathbf{x})}{p_\theta(\mathbf{Z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x})p_\theta(\mathbf{Z}|\mathbf{x})}{q(\mathbf{Z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z}|\mathbf{x})} \right]\end{aligned}$$

A variational characterization of the log likelihood

$$\begin{aligned}\log p_\theta(\mathbf{x}) - D_{KL}(q(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x})) \\&= \log p_\theta(\mathbf{x}) - \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{q(\mathbf{Z}|\mathbf{x})}{p_\theta(\mathbf{Z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x})] - \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{q(\mathbf{Z}|\mathbf{x})}{p_\theta(\mathbf{Z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x})p_\theta(\mathbf{Z}|\mathbf{x})}{q(\mathbf{Z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z}|\mathbf{x})} \right]\end{aligned}$$

- We thus have the following *variational* representation for $\log p_\theta(\mathbf{x})$:

$$\log p_\theta(\mathbf{x}) = \max_{q(\mathbf{z}|\mathbf{x})} \mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z}|\mathbf{x})} \right]$$

Modeling the sampling distribution

- To use *Monte Carlo* sampling to estimate the objective function

$$\mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_{\theta}(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z}|\mathbf{x})} \right]$$

the sampling distribution $q(\mathbf{z}|\mathbf{x})$ must be easy to evaluate and easy to sample from

Modeling the sampling distribution

- To use *Monte Carlo* sampling to estimate the objective function

$$\mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_{\theta}(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z}|\mathbf{x})} \right]$$

the sampling distribution $q(\mathbf{z}|\mathbf{x})$ must be easy to evaluate and easy to sample from

- We thus introduce a second parameterized *model* $q_{\phi}(\mathbf{z}|\mathbf{x})$ and use it to give an *approximate* expression for $\log p_{\theta}(\mathbf{x})$:

$$\log p_{\theta}(\mathbf{x}) \approx \max_{\phi} \mathbb{E}_{\mathbf{Z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_{\theta}(\mathbf{x}|\mathbf{Z})}{q_{\phi}(\mathbf{Z}|\mathbf{x})} \right]$$

Modeling the sampling distribution

- To use *Monte Carlo* sampling to estimate the objective function

$$\mathbb{E}_{\mathbf{Z} \sim q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_{\theta}(\mathbf{x}|\mathbf{Z})}{q(\mathbf{Z}|\mathbf{x})} \right]$$

the sampling distribution $q(\mathbf{z}|\mathbf{x})$ must be easy to evaluate and easy to sample from

- We thus introduce a second parameterized *model* $q_{\phi}(\mathbf{z}|\mathbf{x})$ and use it to give an *approximate* expression for $\log p_{\theta}(\mathbf{x})$:

$$\log p_{\theta}(\mathbf{x}) \approx \max_{\phi} \mathbb{E}_{\mathbf{Z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_{\theta}(\mathbf{x}|\mathbf{Z})}{q_{\phi}(\mathbf{Z}|\mathbf{x})} \right]$$

- Note that in order for the above approximation to be accurate, the model class needs to be sufficiently large so that one can always find a sampling distribution $q_{\phi}(\mathbf{z}|\mathbf{x})$ such that

$$q_{\phi}(\cdot|\mathbf{x}) \approx p_{\theta}(\cdot|\mathbf{x})$$

Importance sampling at work

- The key point here is that when $q_\phi(\cdot|\mathbf{x}) \approx p_\theta(\cdot|\mathbf{x})$, it is *easier* to use Monte Carlo integration to estimate the objective function

$$\mathbb{E}_{\mathbf{Z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q_\phi(\mathbf{Z}|\mathbf{x})} \right]$$

because the expectant

$$\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q_\phi(\mathbf{Z}|\mathbf{x})} \approx \log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{p_\theta(\mathbf{Z}|\mathbf{x})} = \log p_\theta(\mathbf{x})$$

which is *constant* with respect to the random source $\mathbf{Z}$. This is exactly *importance sampling* at work!

Importance sampling at work

- The key point here is that when $q_\phi(\cdot|\mathbf{x}) \approx p_\theta(\cdot|\mathbf{x})$, it is *easier* to use Monte Carlo integration to estimate the objective function

$$\mathbb{E}_{\mathbf{Z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q_\phi(\mathbf{Z}|\mathbf{x})} \right]$$

because the expectant

$$\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{q_\phi(\mathbf{Z}|\mathbf{x})} \approx \log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}|\mathbf{Z})}{p_\theta(\mathbf{Z}|\mathbf{x})} = \log p_\theta(\mathbf{x})$$

which is *constant* with respect to the random source $\mathbf{Z}$. This is exactly *importance sampling* at work!

- Conclusion: We can *jointly* learn an encoder and a decoder by maximizing

$$\frac{1}{m} \sum_{i=1}^m \mathbb{E}_{\mathbf{Z} \sim q_\phi(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p(\mathbf{Z})p_\theta(\mathbf{x}_i|\mathbf{Z})}{q_\phi(\mathbf{Z}|\mathbf{x}_i)} \right]$$

over both θ and ϕ , where the expectation over $\mathbf{Z}$ can be estimated using Monte Carlo integration

Rejection sampling *v.s.* importance sampling

- Finally, let us make an informal comparison between rejection sampling and importance sampling in terms of computing the expectation/integration

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

Rejection sampling *v.s.* importance sampling

- Finally, let us make an informal comparison between rejection sampling and importance sampling in terms of computing the expectation/integration

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

- Let g be a proposal distribution such that the density $\omega(x) = f(x)/g(x) \leq M$ for all $x \in \mathcal{X}$. Such a proposal distribution can be used for both rejection sampling and importance sampling

Rejection sampling *v.s.* importance sampling

- Finally, let us make an informal comparison between rejection sampling and importance sampling in terms of computing the expectation/integration

$$\mathbb{E}_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$$

- Let g be a proposal distribution such that the density $\omega(x) = f(x)/g(x) \leq M$ for all $x \in \mathcal{X}$. Such a proposal distribution can be used for both rejection sampling and importance sampling
- As discussed before, the *self-normalizing* importance sampling estimator is given by

$$\hat{h}'_m = \frac{\sum_{i=1}^m \omega(X_i)h(X_i)}{\sum_{i=1}^m \omega(X_i)}$$

where $(X_1, \dots, X_m)$ are independent samples drawn from the proposal distribution g

- Alternatively, one may select a *subset* of samples $(X_{i_1}, \dots, X_{i_k})$ from $(X_1, \dots, X_m)$ using the *acceptance* rule

$$U_i \leq \frac{\omega(X_i)}{M}$$

- Alternatively, one may select a *subset* of samples $(X_{i_1}, \dots, X_{i_k})$ from $(X_1, \dots, X_m)$ using the *acceptance* rule

$$U_i \leq \frac{\omega(X_i)}{M}$$

- The sub-samples $(X_{i_1}, \dots, X_{i_k})$ are independent samples from f and they can be used to perform classical Monte Carlo integration:

$$\hat{h}_m'' = \frac{1}{k} \sum_{j=1}^k h(X_{i_j}) = \frac{\sum_{i=1}^m \tilde{\omega}(X_i, U_i) h(X_i)}{\sum_{i=1}^m \tilde{\omega}(X_i, U_i)}$$

where

$$\tilde{\omega}(X_i, U_i) = 1_{\left\{U_i \leq \frac{\omega(X_i)}{M}\right\}}$$

- Alternatively, one may select a *subset* of samples $(X_{i_1}, \dots, X_{i_k})$ from $(X_1, \dots, X_m)$ using the *acceptance* rule

$$U_i \leq \frac{\omega(X_i)}{M}$$

- The sub-samples $(X_{i_1}, \dots, X_{i_k})$ are independent samples from f and they can be used to perform classical Monte Carlo integration:

$$\hat{h}_m'' = \frac{1}{k} \sum_{j=1}^k h(X_{i_j}) = \frac{\sum_{i=1}^m \tilde{\omega}(X_i, U_i) h(X_i)}{\sum_{i=1}^m \tilde{\omega}(X_i, U_i)}$$

where

$$\tilde{\omega}(X_i, U_i) = 1_{\left\{U_i \leq \frac{\omega(X_i)}{M}\right\}}$$

- Note the *similarity* between the above two estimators